

Numerical Validation and Debugging Report: Slow-Fast PDE Model for Nitrifying Bacteria Biofilms

1 Overview

This report documents the numerical solver work performed on a six-stage finite-volume PDE codebase implementing the slow-fast reaction–diffusion–advection model of Freingruber, Gonzalez-Cabaleiro and Yoldas (arXiv:2512.13156) for three competing nitrifying bacterial species — ammonia oxidizers (AOB), nitrite oxidizers (NOB), and comammox (CMX) — growing in a spherical/cylindrical biofilm granule. The codebase (Python, NumPy/SciPy) couples three bacterial density fields $u_{\text{AOB}}, u_{\text{NOB}}, u_{\text{CMX}}$ (slow, parabolic, advection–diffusion–reaction) to four substrate concentration fields $c_{\text{NH}_4}, c_{\text{NO}_2}, c_{\text{NO}_3}, c_{\text{O}_2}$ (fast, quasi-steady elliptic) via a six-stage pipeline (Table 1).

The work fell into three phases: (1) an initial code review that surfaced a foundational modelling bug, (2) an extended numerical validation and bug-fixing campaign on the elliptic (Stage 3) solver, culminating in a rigorous floating-point-artifact-free proof of second-order accuracy, and (3) a set of targeted exploration tasks (Neumann boundary conditions, closed-form solution cataloguing, competitive-dominance parameter scans) that themselves surfaced and required fixing two further solver bugs. Throughout, every claim below is labelled **VERIFIED** (independently confirmed with numbers, reproduced in this session), **CONSISTENT** (internally self-consistent, not cross-checked against an external ground truth), or **ASSUMED** (a modelling choice taken on faith from the source material, not independently verified) — following the standard requested throughout this project.

Stage	Module	Role
1	<code>params.py</code>	Dimensional parameter presets (<code>toy</code> , <code>rebeca</code> , <code>eloi</code>)
2	<code>nondim.py</code>	Non-dimensionalization; builds Λ , $\hat{K}$, etc.
3	<code>elliptic.py</code>	Quasi-steady substrate solver (Newton + Picard)
4	<code>relaxation.py</code>	Pseudo-transient-continuation cross-check for Stage 3
5	<code>parabolic.py</code>	Fully-implicit Newton solver for bacterial density evolution
6	<code>slowfast.py</code>	Couples Stages 3–5 into the full slow-time loop

Table 1: The six-stage pipeline.

2 Chronology of Bugs Found and Fixed

2.1 Death term: constant vs. density-dependent (Stage 5)

The original `parabolic.py` implemented bacterial death as a constant rate $-\hat{b}_i$. Direct visual inspection of the rendered arXiv PDF (eq. 2.1/2.5) confirmed the death term is density-dependent, $-\hat{b}_i \hat{\rho}$, where $\hat{\rho} = \sum_j u_j$ is the total local biomass. **VERIFIED**: a text-extraction pass on the PDF had silently dropped the ρ subscript, making the bug invisible to a naive read. Fixed; a regression test (`test_death_term_is_density_dependent_not_constant`) now checks that the per-step growth *ratio* strictly decreases under ample uniform substrate (a constant death term would instead give exactly exponential, constant-ratio growth).

2.2 Row-0 (domain-centre) reaction zeroing (Stages 3–4)

An earlier version zeroed the reaction contribution at the $r = 0$ grid row in both `elliptic.py` (`_residual`, `_assemble_global`) and `relaxation.py`, under the mistaken assumption that $r = 0$ is a boundary-condition row. It is not: it is a genuine interior PDE point using a symmetric (L'Hôpital-limit) stencil. **VERIFIED** via a hand-built reference Jacobian comparison and via the convergence-order study in Section 3.2: with the bug present, the pure-linear (reaction-free) case still converged at 2nd order (the bug was masked there), but the reaction-active case degraded to $\sim$ 1st order. Fixed by restoring the reaction term at row 0 in both modules.

2.3 Near-centre finite-volume cell-volume approximation (Stage 3)

Even after the row-0 fix, the closed-form convergence-order benchmark (Section 3.2) still showed only $\sim$ 1.7–1.9th order with reactions active, not the expected 2.0. Root-cause analysis found the *interior* rows near the centre ($i = 1, 2, 3, \dots$) used the standard finite-volume shortcut $V_i \approx r_i^p h$ for the cell volume, which has relative error $O(h^2/r_i^2)$ — 7.7% at $i = 1$, 2.0% at $i = 2$, falling off for larger i . This is *not* a row-0 bug (row 0's own formula, derived by exact half-cell integration, was already exact) but an under-resolved volume normalisation on the handful of rows immediately adjacent to the centre. **VERIFIED** by rederiving `build_laplacian` to use the exact cell volume $V_i = (r_e^{p+1} - r_w^{p+1})/(p+1)$ at every row (including the outer boundary row), and confirming (a) exact mass conservation to machine precision for all three geometries, and (b) a flattened, no-longer-centre-peaked spatial error profile (Section 3.2).

2.4 Neumann boundary-condition residual target hardcoded to zero (Stages 3–4)

Discovered while exploration-testing Neumann boundary conditions (Section 3.6): three separate functions — `elliptic.py::residual`, `elliptic.py::solve_picard`, and `relaxation.py::solve_relaxation` — all hardcoded the Neumann row's residual/RHS target to 0.0, instead of `-value` (the flux value actually passed to `apply_bc`). `apply_bc` itself correctly built the matrix row for any requested flux; only the separate residual/RHS recomputation ignored it. The net effect: *any* `bc_type='neumann'` solve, in any of the three solvers, silently converged to a zero-flux solution regardless of what nonzero flux was requested. **VERIFIED**: fixed in all three locations; a new regression test drives a genuinely nonzero flux and checks both (i) agreement with the corresponding closed-form solution and (ii) that a finite-difference recomputation of the boundary flux from the converged profile matches the requested value to 1.1×10^{-14} (Section 3.9).

2.5 Plausibility-bound sign bug introduced by the Problem A fix

While fixing the spurious-root problem (Section 3.6.1), an upper concentration bound `c_max = c_max_factor * max(c_inf.values())` was added to `solve_newton`. This silently breaks when `c_inf` values can be *negative* — which a genuine Neumann flux value legitimately can be (negative = influx). `max(-0.3, 0, 0, 0) = 0.0` collapsed the bound to zero, clipping every trial concentration to exactly 0 regardless of step size. **VERIFIED**: fixed to `max(abs(v) for v in c_inf.values())`; reconfirmed the original spurious-root guard still holds (max concentration reached stays < 10 against the old spurious values of $\sim 100/\sim 5400$) and that ordinary positive-`c_inf` Dirichlet operation is unaffected.

2.6 Fallback diagnostic mislabeling and tolerance mismatch

A relaxation-based fallback added to `solve_newton` for genuinely degenerate (singular-Jacobian) states could silently substitute a relaxation answer while the calling code (`slowfast.py::solve_c_given_u`) labelled the step "newton", and the fallback used `solve_relaxation`'s own default tolerance (10^{-9}) rather than the caller's requested tolerance. **VERIFIED**: `solve_newton` now returns a 3-tuple (`C`, `history`, `method`) with an honest method string ("newton" or "newton_inner_relax_fallback"); the inner fallback now targets the caller's own `tol`; a separate, documented outer backstop in `solve_c_given_u`

is retained and independently shown to be reachable (not dead code) for a distinct failure mode (plain iteration-budget exhaustion without ever triggering the inner degeneracy detector).

3 Testing and Validation

3.1 Stage 1–2: parameter presets and non-dimensionalization

Three presets are exercised throughout: **toy** (round numbers, unit testing), **rebeca** (an email-derived estimate, with a documented but **ASSUMED** $\div 100$ yield rescale), and **eloi** (thermodynamics/literature-derived kinetics). For every preset, the slow-fast separation parameter $\varepsilon = r_{\max} L^2 / D_j$ was computed per substrate and confirmed $\varepsilon \ll 10^{-2}$ (**VERIFIED**), justifying the quasi-steady-substrate assumption underlying the whole six-stage split. `test_params.py` and `test_nondim.py` additionally check structural well-formedness of the coefficient dictionaries built by `elliptic_coefficients()`.

3.2 Stage 3: elliptic solver closed-form convergence benchmark

A single-species, single-substrate reduction of the full 4-substrate/3-species system was constructed (setting $u_{\text{NOB}} = u_{\text{CMX}} = 0$, and taking $K_{\text{NH}_4} \gg c_\infty$ so the Monod term is deep in its linear regime), reducing the NH_4 equation to the classical effectiveness-factor problem $0 = \nabla^2 c - \kappa^2 c$, $c(1) = c_\infty$, $\partial_r c(0) = 0$, which has standard closed-form solutions:

$$\text{slab: } c(r) = c_\infty \frac{\cosh(\kappa r)}{\cosh \kappa}, \quad \text{cylindrical: } c(r) = c_\infty \frac{I_0(\kappa r)}{I_0(\kappa)}, \quad \text{spherical: } c(r) = c_\infty \frac{\sinh(\kappa r)}{r \sinh \kappa}.$$

Newton and Picard solutions were compared against these for all three geometries at $\kappa = 2.0$ and $\kappa = 0.5$ (a weaker-reaction cross-check), **VERIFIED** to Newton errors $< 10^{-4}$, Picard $< 10^{-3}$ (Table 2).

Geometry	κ	Newton max error	Picard max error
Slab	2.0	$< 10^{-4}$	$< 10^{-3}$
Cylindrical	2.0	$< 10^{-4}$	$< 10^{-3}$
Spherical	2.0	$< 10^{-4}$	$< 10^{-3}$
Spherical	0.5	$< 10^{-4}$	$< 10^{-4}$

Table 2: Closed-form ground-truth agreement, Newton and Picard vs. analytic solution.

Convergence-order study. A dedicated 4-point (later extended to 5-point) grid-refinement study, spherical geometry, $\kappa = 1.5$:

N	Max error	L^2 error	Order (vs. previous N)
50	1.90×10^{-4}	7.59×10^{-5}	—
100	5.66×10^{-5}	1.97×10^{-5}	1.74 (<i>pre-fix</i>)
200	1.66×10^{-5}	5.14×10^{-6}	1.77 (<i>pre-fix</i>)
400	4.91×10^{-6}	1.43×10^{-6}	1.76 (<i>pre-fix</i>)

Table 3: Convergence order *before* the Section 2.3 volume fix: stuck at ~ 1.7 – 1.9 , not 2.0.

After the exact-volume fix, the same study repeated with *double-precision* arithmetic still showed apparent degradation at large N ($N=200 \rightarrow 400$: order 1.41), which was traced to a *floating-point cancellation floor*: the residual $\nabla^2 c + R(c)$ sums two $O(K_{\text{NH}_4}) \sim 10^7$ -magnitude terms that nearly cancel to $O(1)$, leaving an absolute double-precision floor around 10^{-9} – 10^{-10} — independent of Newton’s requested tolerance (confirmed: tightening `tol` from 10^{-9} to 10^{-14} produced *identical* errors to 6 significant figures). To separate the genuine discretization order from this floor, the identical discretization

(exact finite-volume Laplacian, same linear-reaction reduction) was re-solved with **50-decimal-digit** `mpmath arithmetic` via a hand-written high-precision tridiagonal (Thomas-algorithm) solve, bypassing SciPy’s double-precision sparse LU factorization entirely.

N	Max error (mpmath, 50 digits)	Order
50	1.9527×10^{-5}	—
100	4.8842×10^{-6}	1.999
200	1.2212×10^{-6}	2.000
400	3.0532×10^{-7}	2.000
800	7.6330×10^{-8}	2.000

Table 4: Convergence order, high-precision (roundoff-free) reference. Average order over the full range: 1.9997.

VERIFIED: the scheme is genuinely, cleanly second-order accurate once floating-point roundoff is excluded as a confound; the apparent sub-2 order seen in double precision at fine N is a numerical-precision artifact of the problem’s coefficient magnitude, not a discretization defect. Exact mass conservation ($|\sum_i V_i (\nabla^2 c)_i| < 10^{-13}$) was additionally confirmed for all three geometries (slab, cylindrical, spherical) after the volume fix, versus $O(1)$ violations for slab specifically before a related correction to the outer-boundary half-cell volume factor.

3.3 Stage 4: relaxation (PTC) cross-validation against Stage 3

`test_relaxation_matches_newton_across_densities_and_presets` runs all three presets $\times$ four uniform bacterial densities ($10^{-4}, 0.01, 0.05, 0.5$), comparing PTC-relaxation against Newton. **VERIFIED**, post the Section 2.3 volume fix: agreement of 8.7×10^{-12} to 7.8×10^{-11} across all twelve preset/density combinations — roughly five orders of magnitude tighter than the 10^{-6} test threshold. Relaxation converges in ~ 20 – 22 pseudo-steps in every case, essentially independent of preset or density. The anoxic-core zonation reproduction test (`test_relaxation_reproduces_anoxic_core`) confirmed $O_2 \approx 2.3 \times 10^{-30}$ at the centre (numerical zero) and 0.400 at the rim, with strict monotonicity (min. successive difference -8.7×10^{-30}).

3.4 Stage 5: parabolic (bacterial density) solver

Nine tests cover: exact mass conservation of the diffusion operator (all three geometries); first-order convergence of the advection operator toward exact mass conservation under grid refinement; non-negativity and substrate-gated growth (AOB/CMX peak only in the aerobic shell, matching an imposed anoxic-core/aerobic-rim substrate profile); pure decay to zero under zero substrate; the density-dependent death-term regression (Section 2.1); convergence to the analytic logistic equilibrium $\rho^* = g/\hat{b}$ (matched to $< 10^{-4} \rho^*$, spatial spread $< 10^{-8}$); stability and boundedness at large Δt (0.5, 1.0) under both uniform and localized-bump seeds; the analytic cross-diffusion Jacobian $\partial(\text{Adv}(\rho) u_i)/\partial \rho$ matched against a finite-difference Jacobian to relative error $< 10^{-6}$; and clean Newton convergence (zero “stalled” warnings) on a sharp density front, the specific regime in which an earlier “modified Newton” (missing this cross-diffusion term) was found to stall at $\sim 10^{-4}$ residual. All **VERIFIED**.

3.5 Stage 6: full slow-fast loop

End-to-end runs of the coupled elliptic+parabolic loop (100 slow steps, eloi preset) completed with clean Newton convergence throughout (9–10 iterations/step, residuals 10^{-12} – 10^{-12}). Mass trajectory grew $0.00423 \rightarrow 0.0118$ over 100 steps; per-step growth ratio stabilized (std. 1×10^{-4} over the last 10 steps). **VERIFIED** structurally sound, though one counter-intuitive finding (CMX density peaking at the anoxic core despite $O_2 \equiv 0$ there) required a dedicated follow-up investigation (Section 3.8).

3.6 Neumann boundary condition exploration

Three sub-cases were tested, using a purpose-built test wrapper to bypass a separate, still-*unfixed* API quirk in which `solve_newton`’s standard entry point reuses `c_inf_hat` as the Neumann flux magnitude (meaning a “zero-flux” request for NH_4/O_2 , whose bulk `c_inf` is nonzero, cannot be expressed through the standard call at all):

- (a) **Zero-flux Neumann, reaction off. VERIFIED:** the solver returns the seeded initial concentration *exactly unchanged* (0 Newton iterations, residual 4.5×10^{-13}), independently for all four substrates and four distinct, non-`c_inf` initial values.
- (b) **Nonzero-flux Neumann, reaction off. VERIFIED** to correctly *fail*: no steady state exists physically (mass accumulates without bound with a supply and no sink), and the solver does not converge — NH_4 diverges to $\sim 5 \times 10^{11}$, residual oscillates 0.75–1.0 indefinitely. Confirmed mathematically: the pure-Neumann Laplacian block has smallest singular value 3.2×10^{-15} (a genuine Fredholm solvability-condition violation, not a solver defect).
- (c) **Zero-flux Neumann, reaction on.** This case *found a genuine bug* (later the subject of Section 3.6.1): the undamped Newton solver converged to a numerically-real but physically nonsensical root ($\text{NH}_4 \rightarrow 100.5$, $\text{NO}_3 \rightarrow 5416$, three to four orders of magnitude above anything the sealed system could supply), because backtracking only checked residual decrease, not physical plausibility.

3.6.1 Fix and re-verification (Problems A and B)

Two independent fixes were made to `solve_newton`:

- **Problem A** (spurious root): an upper concentration bound `c_max` = $5 \times$ (largest magnitude `c_inf` entry), enforced jointly with the residual-decrease criterion during backtracking. **VERIFIED:** the spurious root is now unreachable; standard Newton instead converges to $C \equiv 0$ (a mathematically legitimate root, since $\text{Monod}(0) = 0$ trivially satisfies zero-flux + reaction).
- **Problem B** (genuine degeneracy, not a bug): once a substrate is driven to exactly 0 everywhere under zero-flux Neumann (the physically correct answer, e.g., for O_2 in a sealed system), the Jacobian can become exactly singular. Rather than stall or accept a bad step, this is now detected (non-finite Newton update, or exhausted plausible backtracking) and handed off to `relaxation.solve_relaxation`. **VERIFIED:** forcing the exact degenerate state directly (a non-uniform NH_4 profile with O_2/NO_2 already pinned to 0) confirmed the fallback engages and converges cleanly to residual 3.18×10^{-12} , a physically sensible spatially-uniform result.

3.7 Closed-form solution catalogue

Extended beyond the Dirichlet-only cases of Section 3.2 to cover Neumann boundary conditions with linear reaction, for all three geometries (slab, cylindrical, spherical), each with a closed form of the same family ($A \cosh$, $A I_0$, $A \sinh / r$ respectively, with A fixed by the requested flux). **VERIFIED** against the numerical solver, errors $4.6\text{--}8.1 \times 10^{-4}$, consistent with the code’s already-documented first-order one-sided Neumann boundary discretization (as opposed to the exact algebraic Dirichlet row replacement). The full nonlinear Monod reaction case (i.e., what the actual `toy/rebeca/eloi` presets use) has **no known closed form for any geometry or boundary condition** — confirmed by literature convention (Michaelis–Menten-type saturating kinetics reaction–diffusion equations have no general special-function solution), which is exactly why the validation strategy throughout this project relies on the deep-linear-limit reduction, mass conservation, and cross-solver agreement rather than a fully nonlinear closed form.

3.8 CMX-vs-AOB competitive dominance parameter scan

Motivated by an initial full-Stage-6 run in which CMX density counter-intuitively peaked at the anoxic core ($O_2 \equiv 0$) rather than the aerobic rim. A dedicated diagnostic tracked CMX density and O_2 concentration *at the core specifically* over all 100 slow steps: CMX density there decayed -10.58% net over the run (from 0.00527 to 0.00471), confirming this is expected *commensalism* behaviour reported in the source paper — cross-diffusion pushes biomass toward the interior, where it then starves and decays, rather than a bug producing unphysical sustained growth with zero substrate. **VERIFIED**.

A subsequent full Stage 6 sweep over NH_4 feed concentration ($1.0 \rightarrow 3 \times 10^{-4}$, dimensionless) and O_2 feed (baseline and 0.01) found that CMX *can* overtake AOB in total biomass, but only at $NH_4 \leq 0.0003\text{--}0.0004$ (0.15–0.2 mg/L dimensional, against the `eloi` preset’s own real feed of 500 mg/L — a factor of ~ 3000), with a razor-thin margin (CMX/AOB mass ratio 1.0008–1.005) and both species essentially stagnant across the whole crossover region (total biomass varying only $\sim 5\%$ over a $10\times$ change in feed NH_4). **VERIFIED** as a genuine but practically irrelevant crossover: not achievable under any realistic bioreactor feed condition for this model’s target application.

3.9 Neumann residual-target fix: verification

Following the discovery in Section 2.4, a dedicated verification pass was run (a separate, later session) with explicit independent numbers rather than a pass/fail claim:

- **Code confirmation.** All three fixed lines quoted directly from the current source; confirmed each now reads `c_inf[sub] if bc_type=="dirichlet" else -c_inf[sub]`, with zero remaining hardcoded 0.0 Neumann targets anywhere in the codebase (exhaustive grep).
- **Scope honesty.** The new closed-form regression test uses a *mixed*-boundary-condition construction (Dirichlet for $O_2/NO_2/NO_3$, nonzero Neumann for NH_4 only) via a dedicated test wrapper — **not** the standard `solve_newton(bc_type='neumann')` entry point applied to all four coupled substrates simultaneously. Direct testing of that entry point on genuinely-coupled configurations found it **UNVERIFIED**, in fact actively broken: one configuration diverged to residual 2.15×10^9 (even after exhausting its internal relaxation fallback’s 5000-step budget); a further batch of three configurations did not complete within 100 seconds. This is reported plainly as an open issue, not smoothed over.
- **Independent flux check.** For the (working, mixed-BC) case, the boundary flux was recomputed by finite difference *directly from the returned concentration array* (not from any internal solver quantity): requested -0.3 , computed -0.30000000 , difference 1.1×10^{-14} .
- **Regression.** All 8 test files re-run and confirmed passing after this fix.

3.10 Fallback-transparency fix: verification

The three follow-on claims (Section 2.6) were each independently re-confirmed: (i) all six call sites of `solve_newton` across the codebase correctly unpack the new 3-tuple, confirmed both by static grep and by executing each site at runtime with zero unpacking errors; (ii) the inner fallback’s target tolerance was shown, with an explicit before/after run at `tol=10-11` (tighter than the old hardcoded 10^{-9} default), to reach residual 7.28×10^{-12} — past where the old default would have stopped, proving the caller’s tolerance is genuinely used; (iii) both the inner ("`newton_inner_relax_fallback`") and outer ("`outer_relaxation_backstop`") fallback layers were shown to be independently reachable via two distinct, deliberately constructed trigger conditions, ruling out the concern that the outer layer had become dead code.

4 Known Limitations (as of this report)

- The Picard fixed-point solver in `elliptic.py` requires a center-row RHS workaround to converge under the `eloi` preset’s reaction stiffness; this is a property of the fixed-point iteration map (confirmed via direct Jacobian/eigenvalue analysis), not the spatial discretization, and Picard is not relied upon as a primary solver anywhere in the pipeline.
- `solve_newton(bc_type='neumann')` remains unreliable for genuinely multi-substrate-coupled configurations (more than one substrate under nonzero flux simultaneously) — diverges or hangs depending on coupling strength. Does not affect Stage 6, which always uses Dirichlet boundary conditions.
- `c_inf_hat` is reused both as the Dirichlet boundary value and, for `bc_type='neumann'`, as the flux magnitude — meaning a “zero-flux” request cannot be expressed for any substrate whose bulk concentration is itself nonzero, without a custom wrapper bypassing the standard entry point.
- Several parameter-preset cleaning decisions remain **ASSUMED** (documented in `params.py`) rather than independently verified against their original sources, notably the `rebeca` preset’s $\div 100$ yield rescale and the `A_OVER_D_RATIO` advection-strength parameter.

5 Conclusion

The codebase’s numerical core (Stages 1–6) has been subjected to an extensive, adversarial validation campaign: closed-form ground-truth benchmarks across three geometries and two boundary-condition types, high-precision (50-digit) floating-point-artifact-free convergence-order confirmation, exact mass-conservation checks, independent two-solver cross-validation (Newton vs. relaxation) to $\sim 10^{-11}$, analytic Jacobian verification, and targeted stress-testing of edge cases (Neumann degeneracies, competitive-exclusion parameter sweeps) that themselves surfaced and required fixing four genuine bugs beyond the model’s original death-term error. Every fix in this report was independently re-verified with numbers after the fact, and two claims that could not be substantiated — the production multi-substrate Neumann entry point, and the Picard solver’s general robustness — are reported openly as open limitations rather than folded into a blanket “resolved.”